计算机系统基础

许向阳

xuxy@hust.edu.cn

QQ：3303945868

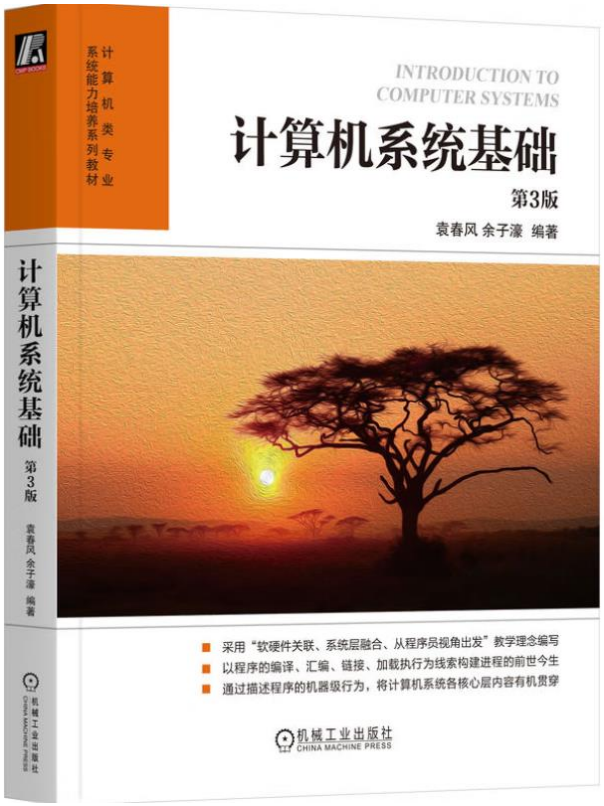
学习资源

QQ：



计算机系统基础_2026年秋_许
群号： 1071335826

学习资源



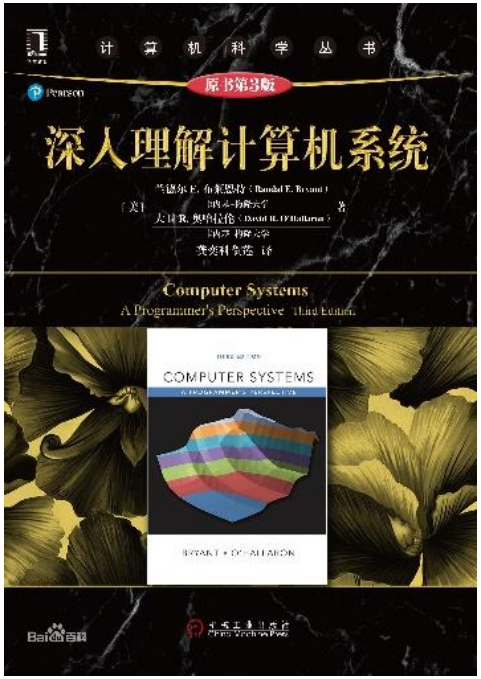
袁春风，余子濠.《计算机系统基础》第3版，机械工业出版社

学习资源

主要参考书

- 《x86汇编语言程序设计》，许向阳，华中科技大学出版社，2020年
- 《深入理解计算机系统》（第3版），Randal E. Bryant, david R. O’ Hallaron著，龚奕利，贺莲 译，机械工业出版社，2016 年





学习资源

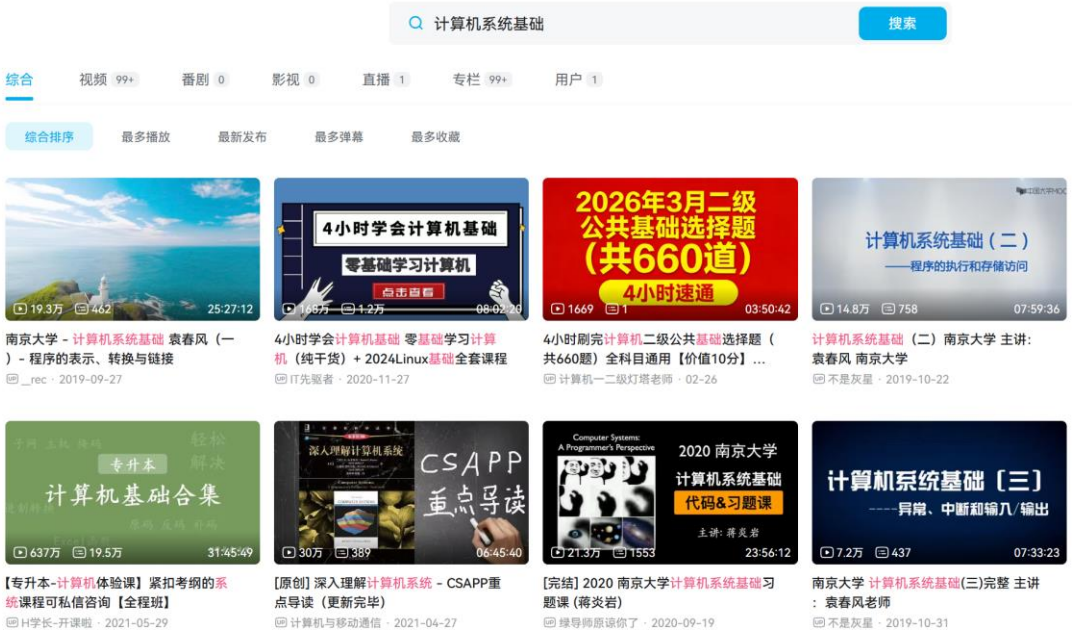
- **MOOC网站（4门系列课程）**
  - https://www.icourse163.org/course/NJU-1001625001
    程序的表示、转换与链接
  - https://www.icourse163.org/course/NJU-1001964032
    程序的执行和存储访问
  - https://www.icourse163.org/course/NJU-1002532004
    异常、中断和输入/输出
  - http://www.icourse163.org/course/NJU-1449521162
    编程与调试实践

学习资源

AI 平台

deepseek　豆包　通义千问　混元

https://www.bilibili.com/



课程考核

- **闭卷考试 ： 占 70%**
- **平时成绩 ： 占 30%**

  考勤： 10%

  作业： 20 %

  作业将在 学校课程平台上发布，在平台中交作业。

  https://smartcourse.hust.edu.cn/

本节课堂目标

- 什么是计算机系统?
- 什么是计算机系统基础?
- 为什么要学习计算机系统基础?
- 学习的主要内容是什么?
- 如何学好这门课?

本节课堂目标

- 了解计算机系统的层次结构，了解各层次之间的关系
- 能够描述一个应用程序开发和运行的全过程
- 能够分析具有相同功能的不同程序的性能差异

  （利用 Cache、单指令多数据流、多线程、编译优化、算法等提高性能）
- 能够指出计算机世界与现实世界信息表示的差异

  （数据表示的长度有限 与 无限的差别）

什么是计算机系统？



Instruction Set Architecture

什么是计算机系统？



计算机视觉、自然语言处理

计算机算法、数据结构

C、C++、JAVA、Python

操作系统　　编译原理

机器语言、汇编语言

计算机组成原理
计算机系统结构

数字逻辑、Verilog语言

模拟电子技术

什么是计算机系统？

全国大学生计算机系统能力大赛

Computer System Development Capability Competition

- 编译系统挑战赛
- 编译系统实现赛
- 操作系统：OS内核实现赛道
- 操作系统： OS功能挑战赛道
- 数据库管理系统设计赛
- CPU设计赛（龙芯杯）

计算机系统基础——学什么？

- 表示（Representation）
  - 数据 如何表示和存储?
    （不同数据类型：带符号整数、无符号整数、浮点数、数组、结构等）
  - 指令 如何表示和编码（译码）？
  - 存储地址（指针）如何表示?
  - 如何生成复杂数据结构中数据元素的地址?
- 转换（Translation）和链接（Link）
  - 高级语言程序对应的机器级代码是怎样的?
  - 可定位/执行目标文件的结构是什么?
  - 如何合并成可执行文件?
  - 程序优化的机理是什么?
- 执行控制流（Control flow）
  - 逻辑控制流中的异常事件及其处理
  - 指令在硬件上的执行过程

为什么要学习“计算机系统基础”？

培养系统思维能力



Q： 程序执行结果
取决于哪些因素？

算法

程序编写

语言处理系统

操作系统

ISA

微体系结构

为什么要学习“计算机系统基础”？

Q：什么是系统思维？

全局思维、动态思维、结构思维



站得更高，看得更远，看得更全！

为什么要学习“计算机系统基础”？

- **系统思维**
  - **从计算机系统角度出发，分析问题和解决问题**
  - **首先取决于对计算机系统有多了解，“知其然并知其所以然”**

基本认识

- **高级语言语句都要转换为机器指令才能在计算机上执行**
- **机器指令是一串0/1序列，能被机器直接理解并执行**
- **计算机系统是模运算系统，字长有限，高位被丢弃**
- **运算器不知道参加运算的是有符号数还是无符号数**
- **在计算机世界，x*x可能小于0，(x+y)+z不一定等于x+(y+z)**
- **访问内存需几十到几百个时钟，而访问磁盘要几百万个时钟**
- **进程具有独立的逻辑控制流和独立的地址空间**
- **过程调用使用栈存放参数和局部变量等，递归过程有大量额外指令，增加时间开销，并可能发生栈溢出**

只有先理解系统，才能驾驭系统、优化系统、研制国产系统!

为什么要学习“计算机系统基础”？

- 强化“系统思维”
- 更好地理解计算机系统，从而编写出更好的程序
- 编写程序时少出错
- 在程序出错时很快找到出错的地方
- 编写出运行更快的程序
- 明白程序是怎样在计算机上执行的
- 为后续课程的学习打下良好基础
- …….

深入理解计算机系统

培养系统性思维能力



东九楼 到 韵苑食堂 路段的交通变迁 的启示

深入理解计算机系统

交通信号与路口优化方案

信号：自适应、绿波、分时段、感应控制、行人按钮

车道：潮汐车道、可变车道、左转待转、
借道左转、直行 + 掉头

结构：渠化、右进右出、行人二次过街、公交优先

看着这些精巧的交通设计，真的会感叹：人类的智慧真的无穷无尽。把混乱变有序，把拥堵变顺畅，把危险变安全，每一个小小的优化，都是用心在解决问题。生活里处处都是这样的智慧，只要愿意观察、愿意思考，总能找到更好的办法。

深入理解计算机系统

计算机作为人类创造的产物，也处处充满人类的智慧！

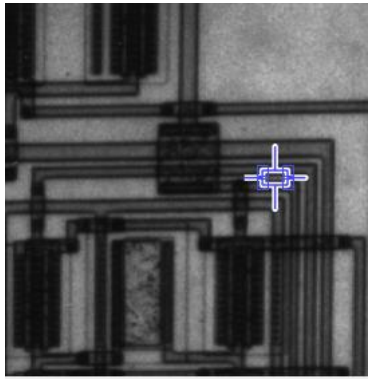
人类在生活中的许多经验都可以用到计算机系统中来。

计算机组成原理、系统结构，根本不是凭空发明的，全是把人类生活里最朴素、最高效的智慧，抽象成了机器规则。

以两个矩阵相减的实现为例，分析程序优化的利用了哪些特性

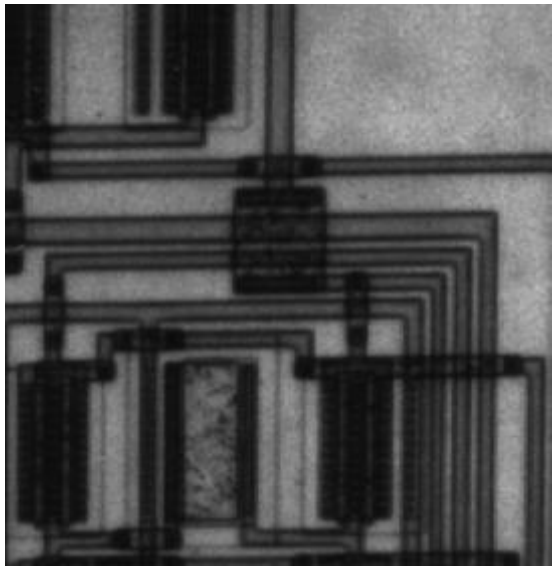
矩阵减法 性能研究

```
int a[2000][3000], b[2000][3000], c[2000][3000];
c= b-a;
```

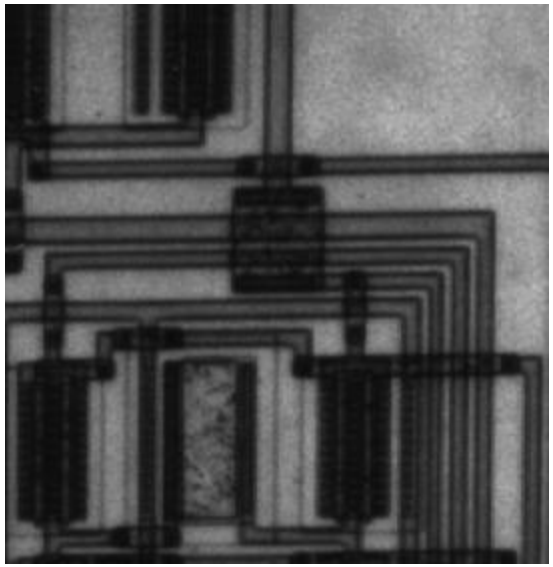


60	59	59	59	58	57	58	59	58	58	55	55	59	59	60	58	55	57	58
48	47	46	46	45	45	44	43	43	44	45	42	45	47	46	46	44	46	46
37	36	35	37	37	35	34	32	32	34	35	35	36	36	35	37	34	36	38
38	39	43	45	45	43	42	39	41	43	42	44	45	43	42	45	42	44	49
57	58	65	67	68	65	64	64	67	68	68	69	72	73	71	68	71	71	77
80	77	81	86	83	79	81	84	91	91	85	83	86	95	97	93	96	95	97
81	79	77	80	79	76	77	79	87	90	79	76	78	85	90	90	87	87	87
59	59	56	56	57	55	54	55	60	62	56	54	56	58	60	60	55	56	57
39	41	41	37	37	40	39	40	41	40	37	36	36	38	38	40	36	36	35
42	45	45	39	41	47	47	46	45	45	43	44	43	43	44	45	43	41	38
59	61	59	55	60	62	62	61	62	62	61	61	61	63	64	62	59	59	57

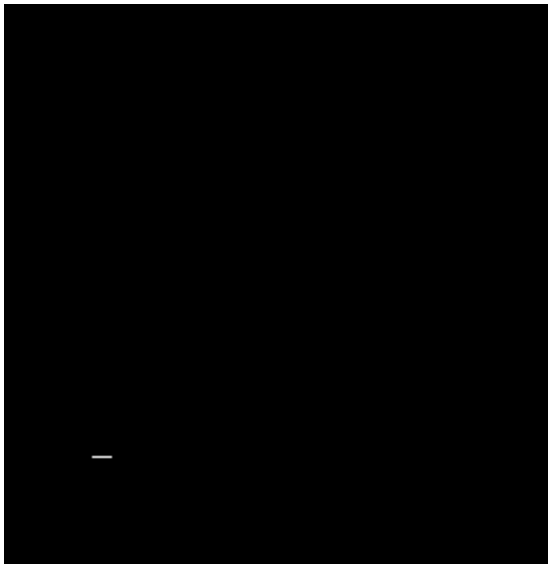
矩阵减法的实际意义



生成的电路板



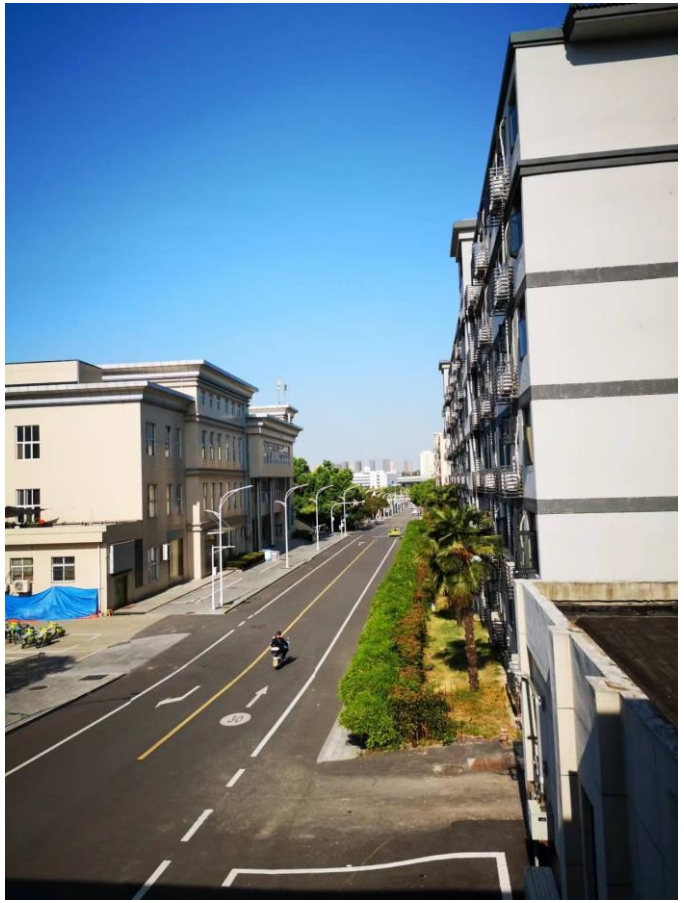
电路 模板

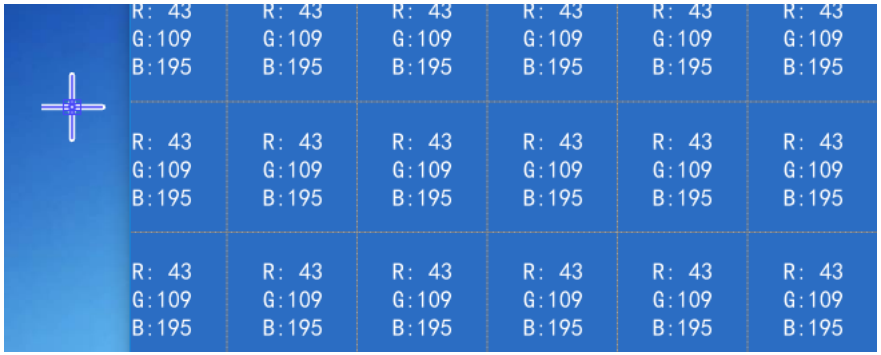


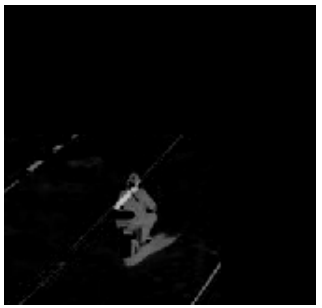
瑕疵 检测

矩阵减法的实际意义









矩阵减法 性能研究

```
void arraysubtract_rowsfirst()
{
    int i, j;
    for (i = 0; i < M; i++)
       for (j = 0; j < N; j++)
           c[i][j] = b[i][j] - a[i][j];
}
```

```
void arraysubtract_colsfirst()
{
    int i, j;
    for (j = 0; j < N; j++)
       for (i = 0; i < M; i++)
           c[i][j] = b[i][j] - a[i][j];
}
```

```
void arraysubtract_onedim()
{
    int i;
    int* pa,*pb,*pc;
    pa = &a[0][0];
    pb = &b[0][0];
    pc = &c[0][0];
    for (i = 0;i < M * N;i++)
        pc[i] = pb[i] - pa[i];
}
```

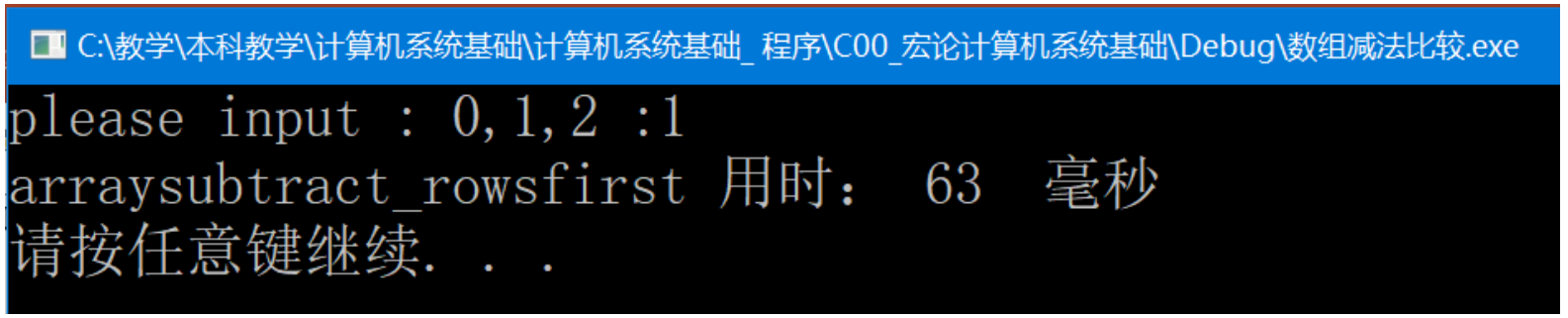
```
#define M 2000
#define N 3000
int a[M][N];
int b[M][N];
int c[M][N];
```

矩阵减法 性能研究

```
int main()
{
    int start, finish,duration,i;
   void (*funcp[3])() = { arraysubtract_colsfirst ,arraysubtract_rowsfirst,
                          arraysubtract_onedim };
   char funcname[3][50] = {"arraysubtract_colsfirst ",
                           "arraysubtract_rowsfirst" , "arraysubtract_onedim" };
   printf("please input : 0,1,2 :");
   scanf("%d", &i);
   start = GetTickCount();  // windows.h。得到系统运行的时间精确到毫秒
   funcp[i]();
   finish = GetTickCount();
   duration = finish - start;
   printf("%s 用时：  %d  毫秒\n", funcname[i],duration);
   system("pause");
   return 0;
}
```

函数指针、函数指针数组、函数入口地址表

矩阵减法 性能研究







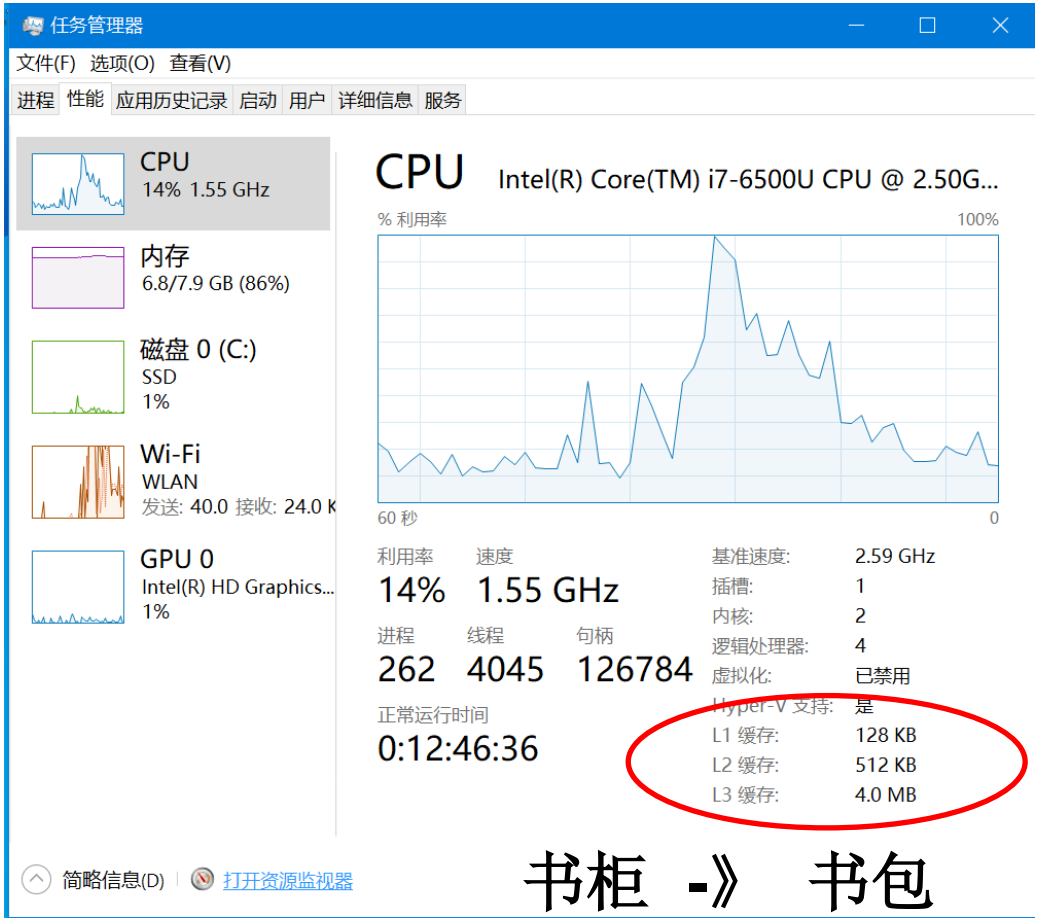
DEBUG 版本下，各函数的运行时间

矩阵减法 性能研究

Q：为什么按行序优先执行快？

矩阵数据元素按行序优先原则存放

第 0 行	第1行	……	……	最后一行



书柜 -》 书包

内存是随机存储器，访问不同位置的数据无时间差异

L1缓存：128KB
L2缓存：512KB
L3缓存：4.0MB

Q：为什么有Cache，行序优先执行快？

矩阵减法 性能研究

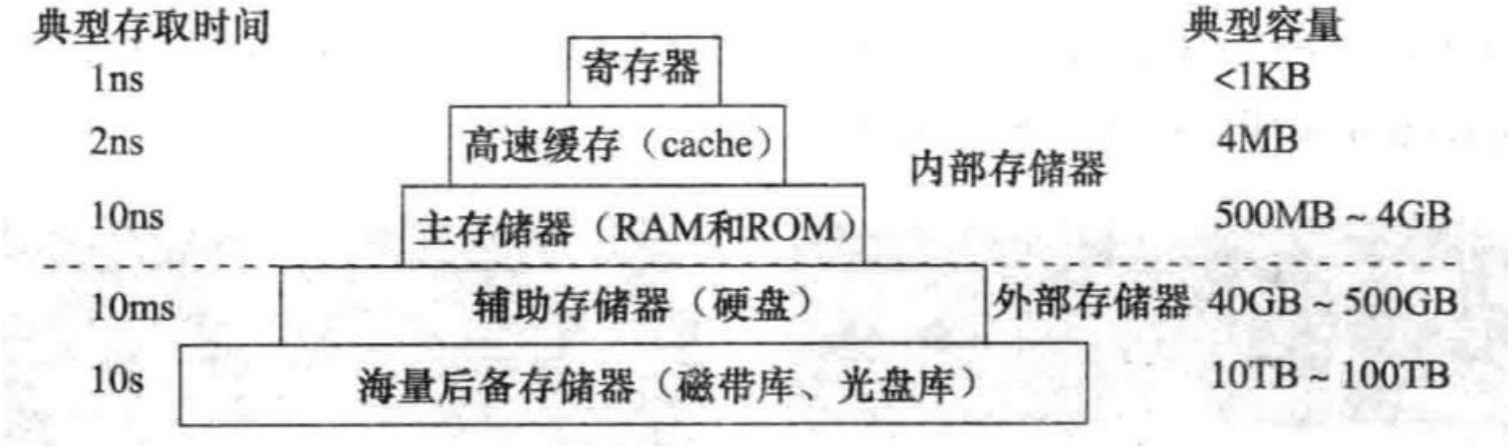
Q：Cache 是什么？ Cache 的工作原理？

为什么Cache 的访问速度快？

Cache 的发展历史？

高速缓冲存储器 (SRAM)，在CPU中
Static Random Access Memory

参考：教材第6章 存储器层次结构
6.4 节 高速缓存



典型容量随时间变化

内存容量不断变化

微型计算机（PC / 笔记本 / 迷你主机）的内存上限，**由系统、CPU、主板共同决定**，2026 年主流与顶配情况如下：

一、理论上限（64 位系统）

64 位地址总线 ：16EB（2^{64}字节） 实际物理地址多为 48 位：256TB

二、Windows 系统限制

家庭版：128GB 专业版：2TB 企业 / 工作站版：6TB

32 位系统：最大 4GB（实际可用约 3.25GB）

三、主流硬件实际上限

1. 台式机（消费级）

中高端主板（DDR5）：128GB–256GB（4 插槽 ×32GB/64GB）

旗舰工作站：512GB–1TB（8 插槽 + 单条 128GB）

2. 笔记本 / 迷你主机

轻薄本：32GB–64GB（2 插槽 / 板载） 游戏本 / 创作本：64GB–128GB

迷你 PC（如雷神 MIX II PRO）：最大 128GB

3. 单条内存容量（2026）

消费级主流：16GB/32GB 高端：64GB/128GB（DDR5）

服务器级：256GB/512GB（价格极高）

普通用户：16GB–32GB够用 游戏 / 创作：64GB起步

顶配微型机：128GB–256GB（消费级） 工作站：512GB–1TB

Cache容量不断变化

CPU 系列	L1（总）	L2（总）	L3	总缓存
i5-13400F	~512KB	20MB	20MB	~40.5MB
R7 9800X	~1MB	8MB	32MB	41MB
i9-14900K	~1.5MB	36MB	36MB	~73.5MB
R9 9950X3D	~2MB	16MB	128MB	146MB

L1 缓存（一级，速度最快）

每核心独立：**32KB–64KB 指令 + 32KB–64KB 数据**

主流 CPU 总 L1：**几百 KB 级别**（如 6 核约 384KB–768KB）

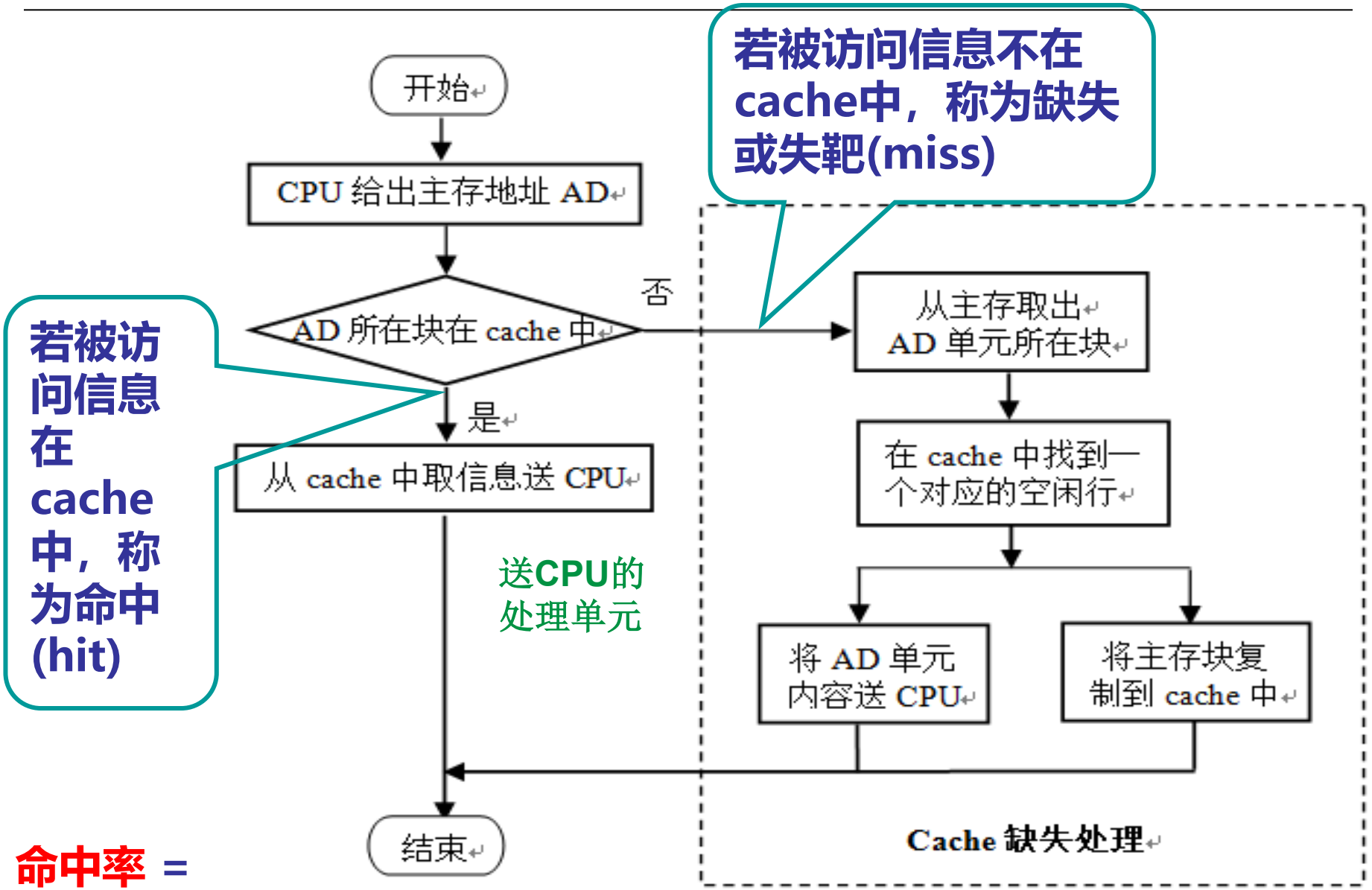
L3 缓存（三级，多核共享，容量最大）

入门 / 主流（i3/R5）：12MB–24MB

中高端（i7/R7）：32MB–64MB

旗舰（i9/R9）：64MB–128MB

矩阵减法 性能研究

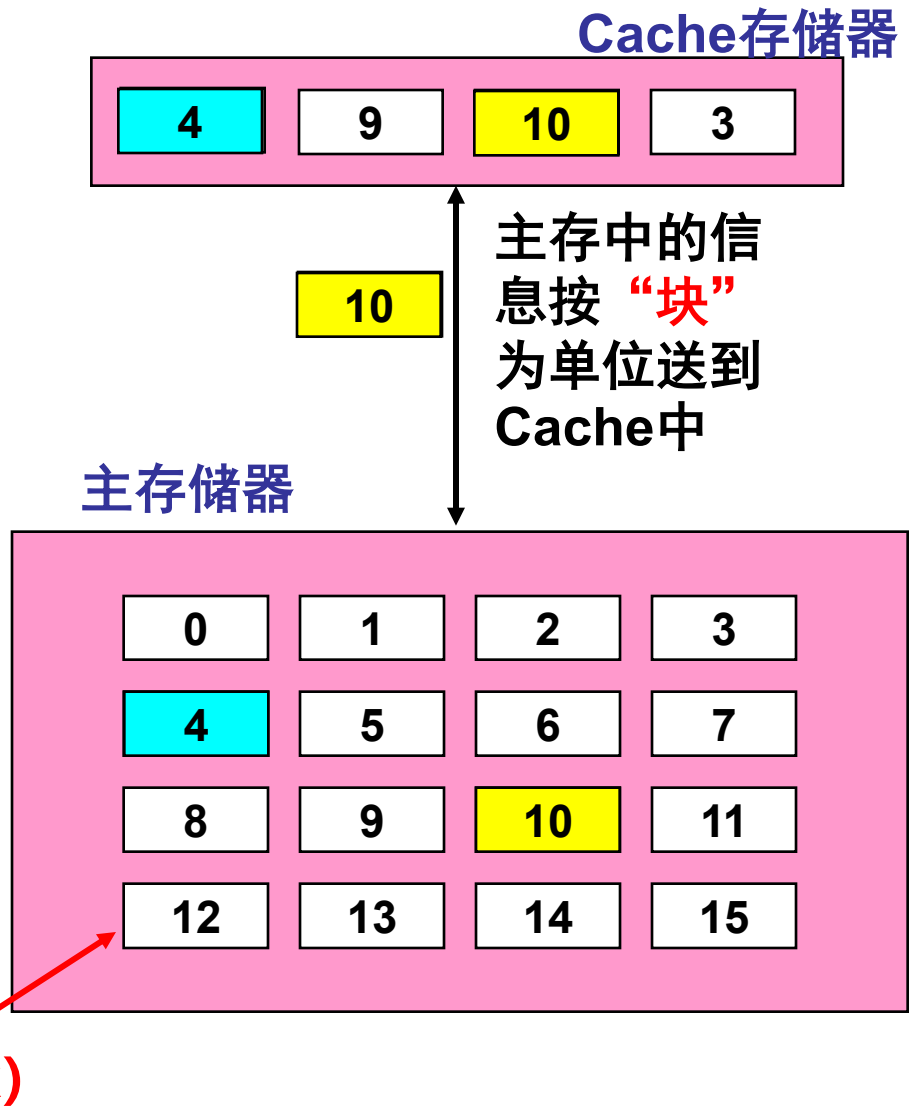


命中率 =
命中次数 / 访问总次数

带cache的CPU访存操作过程

矩阵减法 性能研究

- Cache是一种小容量高速缓冲存储器，它由SRAM组成。
- Cache直接制作在CPU芯片内，速度几乎与CPU一样快。
- 程序运行时，CPU使用的一部分数据/指令会预先成批拷贝在Cache中，Cache的内容是主存储器中部分内容的副本。
- 当CPU需要从内存读(写)数据或指令时，先检查Cache，若有，就直接从Cache中读取，而不用访问主存储器。



矩阵减法 性能研究

问题：要实现Cache机制需要解决哪些问题?

如何分块?

主存块和Cache之间如何映射?

Cache已满时，怎么办?

写数据时怎样保证Cache和MainMemory的一致性?

如何根据主存地址访问到cache中的数据? ……

Cache对程序员(编译器)是否透明? 为什么?

Cache控制器是一个专门设计的硬件模块，通常集成在CPU芯片内部或者作为独立的组件与CPU连接。它负责管理Cache的读取、写入和替换操作，以及主存地址与Cache地址之间的转换。

矩阵减法 性能研究

Q：如何充分利用 cache？

提高Cache 命中率　　减少 Cache 抖动

挖掘程序访问的局部性：

时间局部性、空间局部性

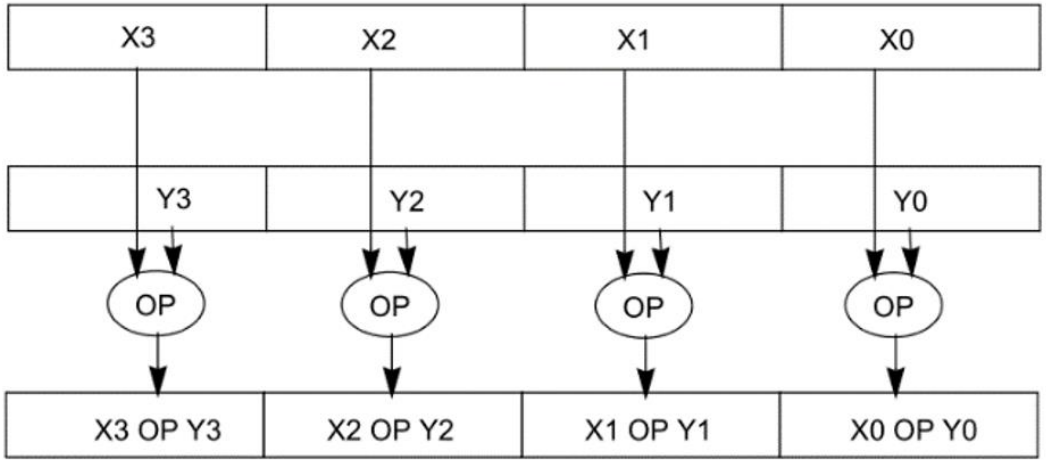
类比：内存 命中率

内存 抖动

矩阵减法 性能研究

CPU 支持 SIMD，能否用 多媒体成组运算指令提高性能？

Single Instruction Multiple Data, 单指令多数据流



	10 (000AH)	20 (0014H)	30 (001EH)	40 (0028H)
+	25 (0019H)	-1 (0FFFFH)	-35(0FFDDH)	35 (0023H)
	35 (0023H)	19 (0013H)	-5 (0FFFBH)	75 (004BH)

矩阵减法 性能研究

Single Instruction Multiple Data, 单指令多数据流

```
#include <emmintrin.h>   //128 位
void arraysubtract_simd_128()
{
    __m128i  *pa ,  *pb,  *pc;
    int LEN4;
    pa = (__m128i*) & a[0][0];
    pb = (__m128i*) & b[0][0];
    pc = (__m128i*) & c[0][0];
    LEN4 = M * N / 4;
    for (int i = 0; i < LEN4; i++) {
        *pc = _mm_sub_epi32((__m128i)*pb, (__m128i)*pa);
        pa += 1;      // 一次 4个成对的整型数相减
        pb += 1;
        pc += 1;
    }
}
```

```
arraysubtract_simd_128 用时:  32  毫秒
```

矩阵减法 性能研究

Single Instruction Multiple Data, 单指令多数据流

```
#include <immintrin.h>   //256 位
void arraysubtract_simd_256()
{
    __m256i *pa, *pb, *pc;
    int LEN8;
    pa = (__m256i*) & a[0][0];
    pb = (__m256i*) & b[0][0];
    pc = (__m256i*) & c[0][0];
    LEN8 = M * N / 8;
    for (int i = 0; i < LEN8; i++) {
        *pc = _mm256_sub_epi32((__m256i)*pb, (__m256i) * pa);
        pa += 1;     // 一次 8个成对的整型数相减
        pb += 1;
        pc += 1;
    }
}
```

```
arraysubtract_simd_256 用时:  16  毫秒
```

矩阵减法 性能研究

CPU 支持 多线程，能否用多线程提高性能？

```
#include <iostream>
#include <thread>
using namespace std;
#define M  2000
#define QM 250  // 2000 /8

void arraysubtract_rowsfirst(int no)
{
    int i, j;
    for (i = QM*no; i < QM*(no+1); i++)
        for (j = 0; j < N; j++)
            c[i][j] = b[i][j] - a[i][j];
    cout << " thread over " << no << endl;
}
```

矩阵减法 性能研究

CPU 支持 多线程，用多线程提高性能

```
int start, finish, duration;
thread subf[8];
init();
start = GetTickCount();

for (int i = 0;i < 8;i++)
  subf[i] = thread(arraysubtract_rowsfirst, i);

for (int i = 0;i < 8;i++)
  subf[i].join();

finish = GetTickCount();

duration = finish - start;
printf("用时:  %d  毫秒\n",  duration);
```

矩阵减法 性能研究

CPU 支持 多线程，用多线程提高性能

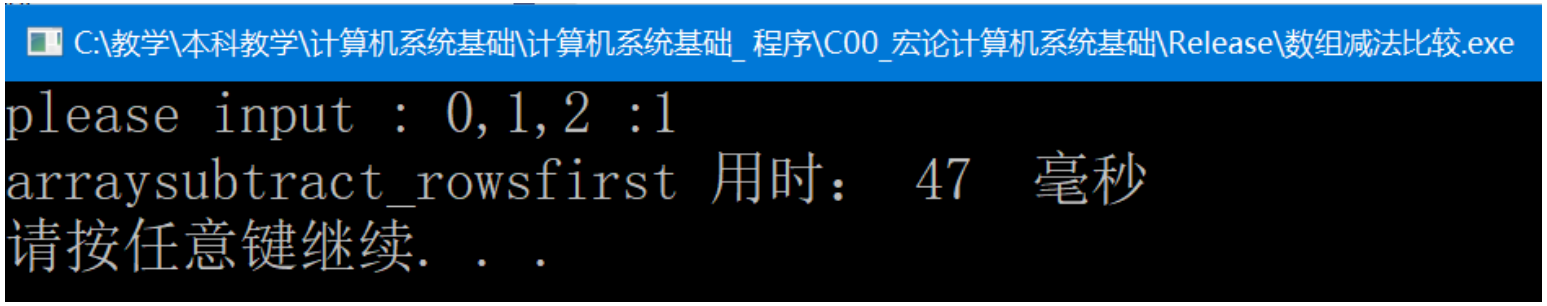
```
thread over 4 thread over 0
thread over 2
thread over 3
thread over 1

thread over 6
thread over 7
thread over 5
用时：  16  毫秒
```

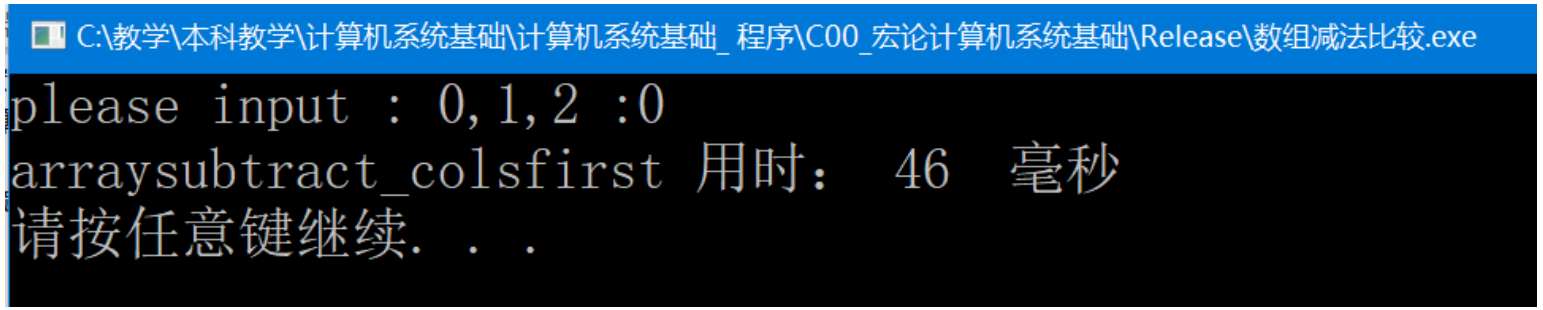
Q：为什么用多线程，能够提高性能？
是不是线程越多越好？

矩阵减法 性能研究

Q: RELEASE 版本下，各函数的运行时间有何变化?



63->47



171->46



78->47

矩阵减法 性能研究

```
movups  xmm0, XMMWORD PTR ?a@@3PAY0HFDA@HA[eax]
movups  xmm1, XMMWORD PTR ?b@@3PAY0HFDA@HA[eax]
psubd   xmm1, xmm0
movups  xmm0, XMMWORD PTR ?a@@3PAY0HFDA@HA[eax+16]
movups  XMMWORD PTR ?c@@3PAY0HFDA@HA[eax], xmm1
movups  xmm1, XMMWORD PTR ?b@@3PAY0HFDA@HA[eax+16]
psubd   xmm1, xmm0
movups  xmm0, XMMWORD PTR ?a@@3PAY0HFDA@HA[eax+32]
movups  XMMWORD PTR ?c@@3PAY0HFDA@HA[eax+16], xmm1
movups  xmm1, XMMWORD PTR ?b@@3PAY0HFDA@HA[eax+32]
psubd   xmm1, xmm0
movups  xmm0, XMMWORD PTR ?a@@3PAY0HFDA@HA[eax+48]
movups  XMMWORD PTR ?c@@3PAY0HFDA@HA[eax+32], xmm1
movups  xmm1, XMMWORD PTR ?b@@3PAY0HFDA@HA[eax+48]
psubd   xmm1, xmm0
movups  XMMWORD PTR ?c@@3PAY0HFDA@HA[eax+48], xmm1
add     eax, 64                                  ; 00000040H
sub     ecx, 1
jne     SHORT $LL7@arraysubtr
```

优化：按行序执行；双重循环展开为单循环；使用 SIMD 指令

矩阵减法 性能研究

总结：

- 按行序比按列序处理要快

  利用矩阵数据按行序存储的特性，提高了 CPU 中的 cache 的命中率，减少了cache 抖动。

- 使用SIMD 指令更快

  成组运算，减少了运算次数。

- 使用多线程更快

  单核多线程（并发）、多核多线程（并行）

- 编译优化 ——调整执行顺序、循环展开、使用SIMD指令

深入理解计算机系统

仅仅依靠编译器的优化，能否包打天下？

Release 版是否一定比Debug 版快？

Release 版是否一定能优化得很好？

深入理解计算机系统

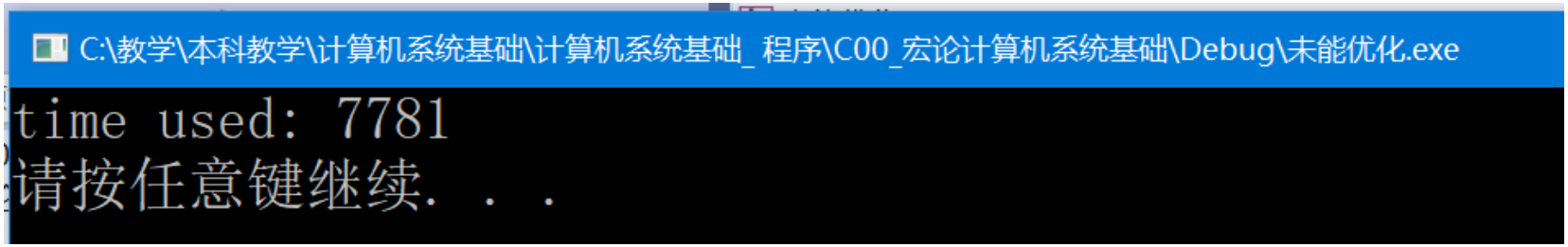
一个字符串中小写字母变成大写字母

```
void change()
{
    int i;
    int start, finish, duration;
    start = GetTickCount();
    for (i = 0;i < strlen(block); i++) {
        if (block[i] >= 'a' && block[i] <= 'z')
            block[i] = block[i] - 'a' + 'A';
    }
    finish = GetTickCount();
    duration = finish - start;
    cout <<"time used: "<< duration << endl;
}
```

```
#define SIZE  4096*50
char  block[SIZE];

block[SIZE-10]=0;
```

深入理解计算机系统





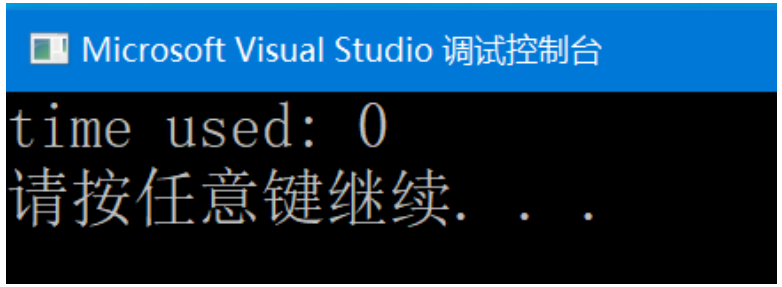
Debug　：7781 毫秒　　Release：19765 毫秒

编译优化不是万能的。对于写得不好的程序，编译器不敢优化，甚至生成性能更差的程序。

深入理解计算机系统

```
void change()
{
    int i;
    int start, finish, duration;
    start = GetTickCount();
    for (i = 0;i < strlen(block); i++) {
        if (block[i] >= 'a' && block[i] <= 'z')
            block[i] = block[i] - 'a' + 'A';
    }
    finish = GetTickCount();
    duration = finish - start;
    cout <<"time used: "<< duration << endl;
}
```

为何慢？如何优化?




```
int  len = strlen(block);
for (i = 0;i < len; i++) { …… }
```

此处的算法优化，重复的操作只做一次，并未利用硬件的特性

深入理解计算机系统

求整型数组中len个元素的和

```
int sum_1(int a[ ], unsigned int len)
{
    int   i，sum = 0;
    for   (i = 0; i <= len–1; i++)
          sum += a[i];
    return sum;
}
```

```
int main()
{
    int   a[3]={10,20,30};
    int result;
    result=sum_1(a,0);
    printf(“%d”,result);
}
```

Q：运行时会看到什么？

深入理解计算机系统

```
6  int sum_1(int a[], unsigned int len)
7  {
8      int      i,sum = 0;
9      for (i = 0; i <=len-1; i++)
10         sum += a[i];
11     return sum;
12 }
```

已引发异常

引发了异常: 读取访问权限冲突。
a 是 0x18D4112。

复制详细信息 | 启动 Live Share 会话...

▲ 异常设置
- ☑ 引发此异常类型时中断
  从以下位置引发时除外:
- ☐ 运算等价性.exe

打开异常设置 | 编辑条件

```
19     return sum;
20 }
```

监视 1

搜索(Ctrl+E)　搜索深度: 3

名称	值	类型
a[0]	10	int
a[1]	20	int
a[2]	30	int
i	4492	int
sum	1519703788	int
len	0	unsigned int

Q：什么是“读取访问权限冲突”？
为什么会有“读取访问权限冲突”？
“访问权限”的实现机理是什么？
为什么会有此现象？

深入理解计算机系统

```
int sum_1(int a[ ], unsigned int len)
{
    int   i，sum = 0;
    for   (i = 0; i <= len–1; i++)
          sum += a[i];
    return sum;
}
```

```
int main()
{   int   a[3]={10,20,30};
    int result;
    result=sum_1(a,0);
    printf(“%d”,result);
}
```

编译时：warning C4018: “<=”：有符号/无符号不匹配

分析：0－1 的结果是 0xFFFFFFFF

一个有符号变量和一个无符号变量，按照无符号数进行比较。

i =0、1、2 …… <= 0xFFFFFFFF

依次访问 a[0]、a[1]、a[2]、a[3]、a[4]、……

计算 a[i] 地址的机器指令 ---- 编译系统

程序所占的空间范围管理 ---- 操作系统

执行指令时的地址空间检查 ---- CPU 引发异常

访问地址异常处理 ---- 操作系统

深入理解计算机系统

操作系统：内存管理

计算机组成原理：中断和异常

```
#include <iostream>
#include <Windows.h>
using namespace std;
int sum_1_exception(int a[], unsigned int len)
{
    int i, sum = 0;
    __try {
        for (i = 0; i <= len - 1; i++)
        sum += a[i];
    }
    __except(EXCEPTION_EXECUTE_HANDLER )
    {
        cout << "exception occur" << endl;
        sum = 0;
    }
    return sum;
}
```

深入理解计算机系统

```
int sum_1(int a[ ], unsigned len)
{
    int   i，sum = 0;
    for   (i = 0; i <= len–1; i++)
          sum += a[i];
    return sum;
}
```

```
int sum_2(int a[ ], unsigned len)
{
    int   i，sum = 0;
    for   (i = 0; i < len; i++)
          sum += a[i];
    return sum;
}
```

```
int main()
{
    int   a[3]={10,20,30};
    int result;
   // result=sum_1(a,0);
    result = sum_2(a,0);
    printf(“%d”,result);
}
```

改为右边的形式后，运行正常。显示结果0

i<=len-1 ⇔ i<len ？

x<y ⇔ x-y<0 ？

数学世界与计算机世界有差异！

深入理解计算机系统

```
int sum_1(int a[ ], unsigned len)
{
    int   i，sum = 0;
    for   (i = 0; i <= len–1; i++)
          sum += a[i];
    return sum;
}
```

```
int sum_2(int a[ ], unsigned len)
{
    int   i，sum = 0;
    for   (i = 0; i < len; i++)
          sum += a[i];
    return sum;
}
```

result=sum_3(a,0);运行正常

```
int sum_3(int a[ ], int len)
{
    int   i，sum = 0;
    for   (i = 0; i <= len-1; i++)
          sum += a[i];
    return sum;
}
```

result=sum_3(a,0x90000000);
返回的结果为0

result=sum_1(a,0x90000000);
result=sum_2(a,0x90000000);
运行异常

有符号数(int)，无符号数(unsigned int)

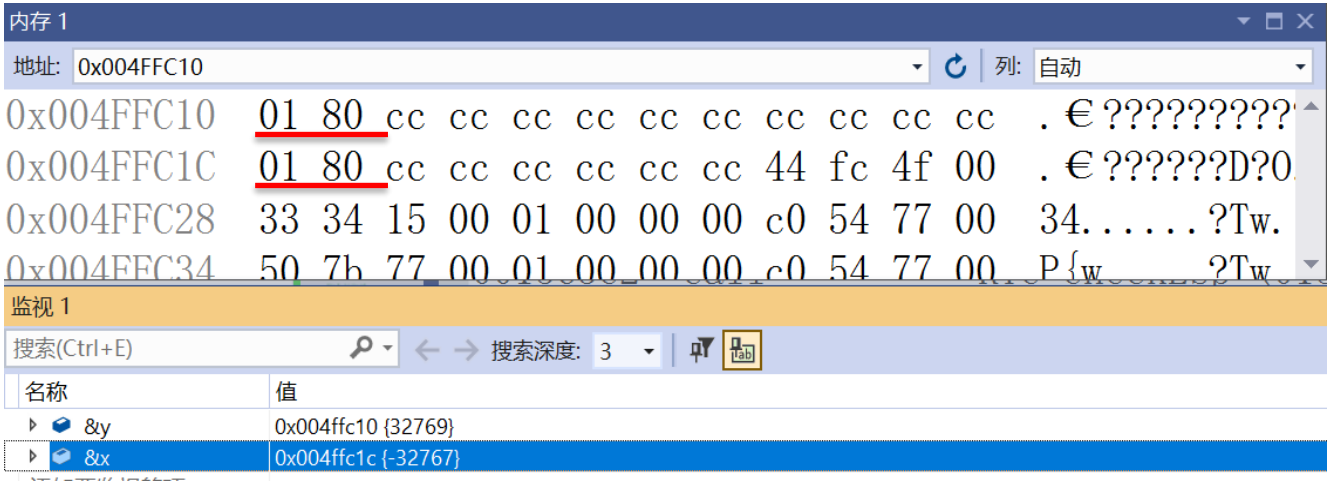
测验

```
short x = 0x7fff;
unsigned short y = 0x7fff;
cout << "x=" << x << "  y=" << y << endl;
x++;       y++;
cout << "x=" << x << "  y=" << y << endl;
x++;       y++;
cout << "x=" << x << "  y=" << y << endl;
cout << "x=" << (unsigned short)x << "  y=" << (short)y << endl;
```



运行结果是什么?

0x7ffff +1 -> 0x 8000
0x8000+1 -> 0x 8001



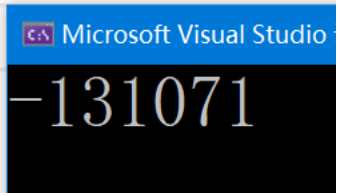
相同的内存内容，用不同的方式去解读，得到不同的结果！

测验

若x和y为int型， 当x=65535时， y=x*x; y的值为多少?

```
#include <iostream>
using namespace std;

int main()
{
    int x = 65535;
    int y = x * x;
    cout << y << endl;
    return 0;
}
```



现实世界中，$x^2 \geq 0$，但在计算机世界并不一定成立。

测验

```
void f()
{
    int x = -2147483648;
    int y = 1;
    if (x < y)
        cout << "x<y" << endl;
    if (-x > -y)
        cout << "-x>-y" << endl;
    cout << "x=" << x <<"  -x="<<-x<< endl;
}
```

```
x<y
x=-2147483648  -x=-2147483648
```

x < y ⇔ -x > -y

在现实世界中成立，但在计算机世界中并不一定成立。

理解该问题需要知道：

机器级数据的表示

机器指令的执行

x= 0x 80 00 00 00;

测验

main.c

```
int d=100;
int x=200;
int main()
{
    p1( );
    printf ( “d=%d, x=%d\n” , d, x );
    return 0;
}
```

p1.c

```
double d;

void p1( )
{
    d=1.0;
}
```

打印结果是什么?

d=0，x=1 072 693 248

Why?

理解该问题需要知道：

机器级数据的表示

变量的存储空间分配

数据的大端/小端存储方式

链接器的符号解析规则

……

测验

以下是一段C语言代码：

```
#include <stdio.h>
main()
{
    double a = 10;
    printf("a = %d\n", a);
}
```

理解该问题需要知道：

IEEE 754 的表示

X87 FPU的体系结构

IA-32和x86-64中过程调用的参数传递

计算机内部的运算电路

……

在IA-32上运行时，打印结果为a=0

在x86-64上运行时，打印出来的a是一个不确定值

为什么?

测验

```
double fun(int i)
{
  volatile double d[1] = {3.14};
  volatile long int a[2];
  a[i] = 1073741824; /* Possibly out of bounds */
  return d[0];
}
```

i=0~4时，fun(i)分别返回什么值?

fun(0) → 3.14
fun(1) → 3.14
fun(2) → 3.1399998664856
fun(3) → 2.0000006103515 6
fun(4) → 3.14, 然后存储保护错

Why?

理解该问题需要知道：

机器级数据的表示

过程调用机制

栈帧中数据的布局

……

测验

C/C++ code

```
#include "stdafx.h"
int main(int argc, char* argv[])
{
    int a=10;
    double *p=(double*)&a;
    printf("%f\n",*p);                  //结果为0.000000
    printf("%f\n",(double(a)));         //结果为10.000000

    return 0;
}
为什么printf("%f",*p)和printf("%f",(double)a)结果不一样呢？
```

理解该问题需要知道：

数据的表示

编译（程序的转换）

局部变量在栈中的位置

……

不都是强制类型转换吗？怎么会不一样

关键差别在于一条指令：

fldl 和 fildl

总 结

理解程序的执行结果要从系统层面考虑！

学完“计算机系统基础”就会对计算机系统有清晰的认识，
以后再学其他相关课程就容易多了。

把许多概念和知识联系起来就是李国杰院士所提出的“系统思维”。
即：站在“计算机系统”的角度考虑问题！

思考

本课程的主要内容有哪些?

什么是 Cache 命中率? 如何提高Cache的命中率?

在两个二维数组相减 的计算中，采用了哪些策略来提高性能?
各个提高性能的策略对应的机理是什么?

在什么情况下，数组越界访问能被 CPU 检测出来?
在什么情况下不能检测出来?

ISA 的英文全称是什么？ 它是什么子系统之间的接口?

能否举出一个 I/O 抖动的程序例子?